Studio Nx

백엔드 264 — 프로젝트 전체 응답 스키마

2026-10-19

Backend(FastAPI/Python) 264 — 프로젝트 전체 응답 스키마 (`full_project.py`)

항목	내용
파일	`studionx/backend/app/schemas/full_project.py`
작업	S1-10 프로젝트 전체 저장·불러오기: 전체 조회와 곡 설정 스키마 (PRD 02 FR-253, FR-254)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전 · 다음	263 `wireCells` 외 화면 배선 · 265 `_group_by` · `project_info`

1. 하는 일

시퀀서가 화면을 열 때 받는 한 덩어리 응답의 모양을 정합니다. 네 클래스를 한 파일에 둡니다.

```
GET /projects/{id}/full
└ FullProjectResponse
   ├ project  FullProjectInfo      곡 설정까지 담은 프로젝트 (bpm · 박자 · Key · master_settings · revis
ion)
   ├ tracks[] FullTrackResponse    트랙 한 줄 + instruments · effects · links
   ├ patterns[] PatternResponse    패턴 + notes (S1-08 스키마를 그대로 씁니다)
   └ clips · markers · tempo_changes · meter_changes   지금은 늘 빈 목록
```

곡 설정을 저장하는 요청의 모양(FullProjectSettingsUpdateRequest)도 여기 있습니다.

2. 언제 쓰나

쓰는 곳	
시퀀서 `sequencer.js` `loadProject`	`GET /projects/{id}/full` 의 응답을 이 모양으로 검사해 받습니다
시퀀서 수동 저장·자동 저장	바뀐 곡 설정만 `PATCH /projects/{id}/settings` 로 보냅니다
백엔드 라우터·서비스	`full_project_service` 가 이 클래스들을 채워 돌려줍니다

3. 코드

```
class FullProjectInfo(BaseModel):
    project_id: int
    title: str
    description: str | None
    bpm: Decimal
    time_sig_num: int
    time_sig_den: int
    music_key: str | None
    scale_mode: str | None
    sample_rate: int
    master_settings: dict[str, Any] | None
    revision: int
```

```
    my_role: str
    member_count: int
    opened: bool
    created_at: datetime
    updated_at: datetime


class FullTrackResponse(TrackResponse):
    instruments: list[TrackInstrumentResponse]
    effects: list[TrackPluginResponse]
    links: list[TrackPluginLinkResponse]


class FullProjectResponse(BaseModel):
    project: FullProjectInfo
    tracks: list[FullTrackResponse]
    patterns: list[PatternResponse]
    clips: list[dict[str, Any]] = Field(default_factory=list)
    markers: list[dict[str, Any]] = Field(default_factory=list)
    tempo_changes: list[dict[str, Any]] = Field(default_factory=list)
    meter_changes: list[dict[str, Any]] = Field(default_factory=list)


class FullProjectSettingsUpdateRequest(BaseModel):
    bpm: Decimal | None = Field(default=None, ge=BPM_MIN, le=BPM_MAX, decimal_places=2)
    time_sig_num: int | None = Field(default=None, ge=1, le=TIME_SIG_MAX)
    time_sig_den: int | None = Field(default=None, ge=1, le=TIME_SIG_MAX)
    music_key: str | None = Field(default=None, max_length=10)
    scale_mode: str | None = Field(default=None, max_length=20)
    sample_rate: int | None = Field(default=None, ge=SAMPLE_RATE_MIN, le=SAMPLE_RATE_MAX)
    master_settings: dict[str, Any] | None = None
```

문법 풀이

식	뜻
`class FullTrackResponse(TrackResponse)`	트랙 칸(231)을 그대로 물려받고 장치 세 목록만 더합니다. 트랙 칸 이름이 목록 API 와 어긋나지 않습니다
`Field(default_factory=list)`	기본값을 빈 목록으로 둡니다. `[]` 를 그대로 쓰면 같은 목록을 모든 응답이 나눠 쓰게 되어 위험합니다
`Decimal | None = Field(default=None, ge=..., le=...)`	보내지 않으면 `None` 이고, 보내면 범위를 검사합니다
`BPM_MIN` · `BPM_MAX`	20 과 300 입니다. 파일 위쪽에 상수로 두어 시험과 서비스가 같은 값을 봅니다
`master_settings: dict[str, Any] | None`	자유로운 모양의 마스터 설정 묶음입니다 (DB 칸이 JSON)

4. 왜 이렇게

선택	이유
응답을 한 클래스에 중첩	화면이 여섯 번 요청하던 것을 한 번으로 줄입니다 (FR-254)
트랙 스키마를 물려받음	트랙 칸 이름을 새로 적으면 목록 API 와 어긋날 수 있습니다

선택	이유
아직 없는 표는 빈 목록으로 내보냄	화면이 `clips` · `markers` 칸을 늘 있다고 보고 읽을 수 있습니다. 나중에 S2-04(오토메이션)와 클립 저장 작업이 이 목록만 채웁니다
`revision` 을 함께 내보냄	나중에 동시 편집(S3-02)이 어긋남을 알아채는 값입니다. 지금은 화면이 읽기만 합니다
곡 설정 수정은 보낸 칸만	화면이 Ctrl+S·자동 저장 때 바뀐 칸만 보내므로 나머지는 건드리지 않습니다

5. 시험

`tests/test_full_project_routes.py` 에서 응답의 칸 이름 집합(PROJECT_KEYS · TRACK_KEYS · PATTERN_KEYS · NOTE_KEYS)과 빈 목록 네 칸을 확인했습니다. `tests/test_project_settings_routes.py` 에서 범위를 벗어난 BPM(5 · 400) · 박자(0) · 샘플레이트(100)가 422 로 막히는지 확인했습니다. 모두 통과합니다.